(FINAL YEAR MINI PROJECT ARCHITECTURE & WORKFLOW)

Hybrid AI-Assisted Page Replacement: System Architecture and Workflow

---

1) High-level summary (one line)

When the OS experiences a page fault, it uses the I/O wait time to ask an AI predictor (GRU/LSTM) to produce a short, ordered multi-step prediction chain. The OS keeps running LRU as the fast critical path; the AI acts as an advisor whose suggestions are accepted only when a dynamic trust score (fusing per-page fuzzy μ, short term S, and long-term reality W) is high. Predictions are validated step-by-step and the system updates page-level and global trust using rewards/penalties.

---

2) Core components (names & roles)

1. **OS / Memory Manager (LRU)** — critical path, O(1) eviction; issues page faults and asks for prediction suggestions.
2. **AI Predictor (Embedding + GRU/LSTM)** — produces ordered, multi-step future page predictions (one-shot during I/O wait).
3. **Predicted Queues**
    a. **Old chain q**: currently-active predicted sequence (may have been generated earlier).
    b. **New chain p**: chain produced on most recent page fault.
4. **Waiting / Hold Queue** — pages the model advises OS to "hold" (defer eviction) temporarily.
5. **Trust Controller** — computes final trust from μ (per-page fuzzy), S (short term model self-score), W (real-world reality weight).
6. **Reality Daemon** — background process that slowly updates W using hold-suggestion success statistics (EMA).
7. **Metrics** — page fault rate, hit rate, AMAT, hold suggestion success ratio.

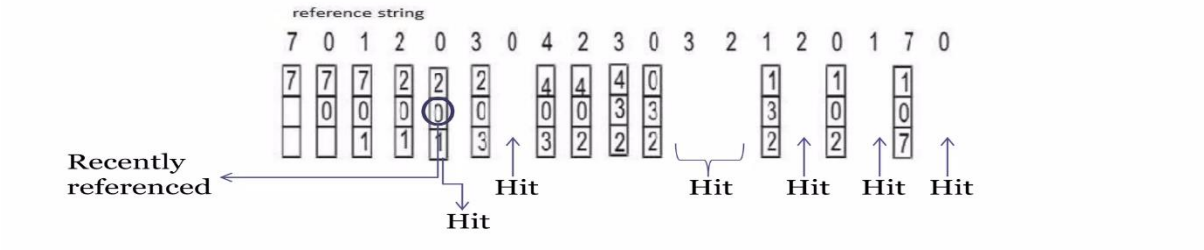
---

3) Data structures & initial values (explicit)

Use these names and types in explanations / code:

- history (list of last N page IDs) — input to model.
- p (list) — new predicted chain produced by model.
- q (list) — old/active predicted chain.
- hold_queue (set/list) — pages marked "hold" by the model.
- mu[page] ∈ [0,1] — fuzzy membership per page (initial default: 0.8).
- S ∈ [0,1] — short-term accuracy/confidence (initial default: 0.8).
- W ∈ [0,1] — reality weight (initial default: 0.7).
- Tuning constants (suggested):
    - mu_reward_r = 0.10 (reward factor), mu_penalty_p = 0.30 (penalty factor).
    - S_alpha = 0.15 (fast update for short-term).
    - W_alpha = 0.10 (daemon EMA learning rate).
    - trust_threshold — final threshold for OS to obey suggestion (e.g. 0.6–0.8).

(These numbers are suggestions — tune in experiments.)



4) When the model runs (activation policy)

- **Model runs only on page fault**: when OS experiences a page fault and starts disk/SSD I/O, model uses that I/O wait time to generate a full prediction chain (one-shot multi-step). This avoids running the model on the critical path.
- **Chain length** is adaptive:
- chain_len = min(max_chain, floor(IO_time / model_time_per_prediction))

Example: if IO_time = 100 µs, model_time_per_pred ≈ 5 µs → chain_len ≈ 20.

5) What the model outputs

- Ordered predicted pages: p = [p1, p2, p3, ...] (head is immediate next).
- Per-step confidences conf = [c1, c2, ...] (sigmoid outputs — fuzzy confidences).
- Optionally: model also outputs cp (immediate next) and long chain, and can produce per-prediction µ proposals.

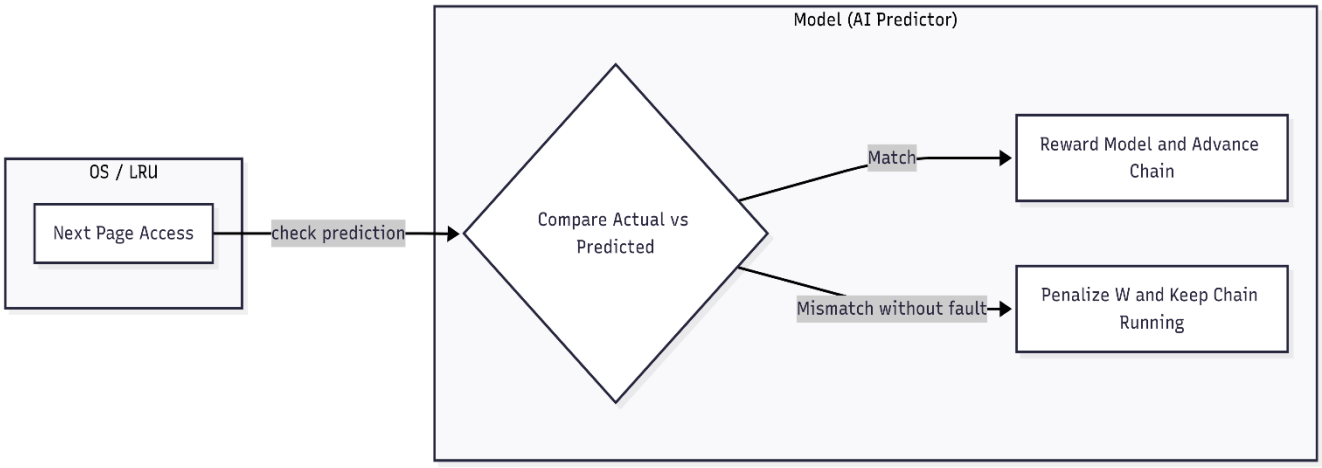
Model inputs: history, mu values for pages in history, and W (global reality weight) and S optionally.
Implementation note: prefer **GRU** for faster inference with close accuracy; do multi-head output in one forward pass.

6) Continuous chain usage & per-access behavior

For every **incoming page access** (normal CPU access), follow:

1. If q (active chain) exists → compare actual page c with q[0] (head).
    - If c == q[0] → **Match**: reward model, advance q (pop head). Update mu[c] and S. No LRU involvement.
    - If c != q[0] but page is in memory (no page fault) → **Mismatch without fault**: **do NOT discard the chain**; *soft-penalize* model trust W (or S) and continue (advance/adjust queues as described below).
2. If a **true page fault** occurs (page not in memory) → follow Page Fault Handling (next section).

This keeps the chain “alive” across soft mismatches; only a true page fault causes regeneration.



---

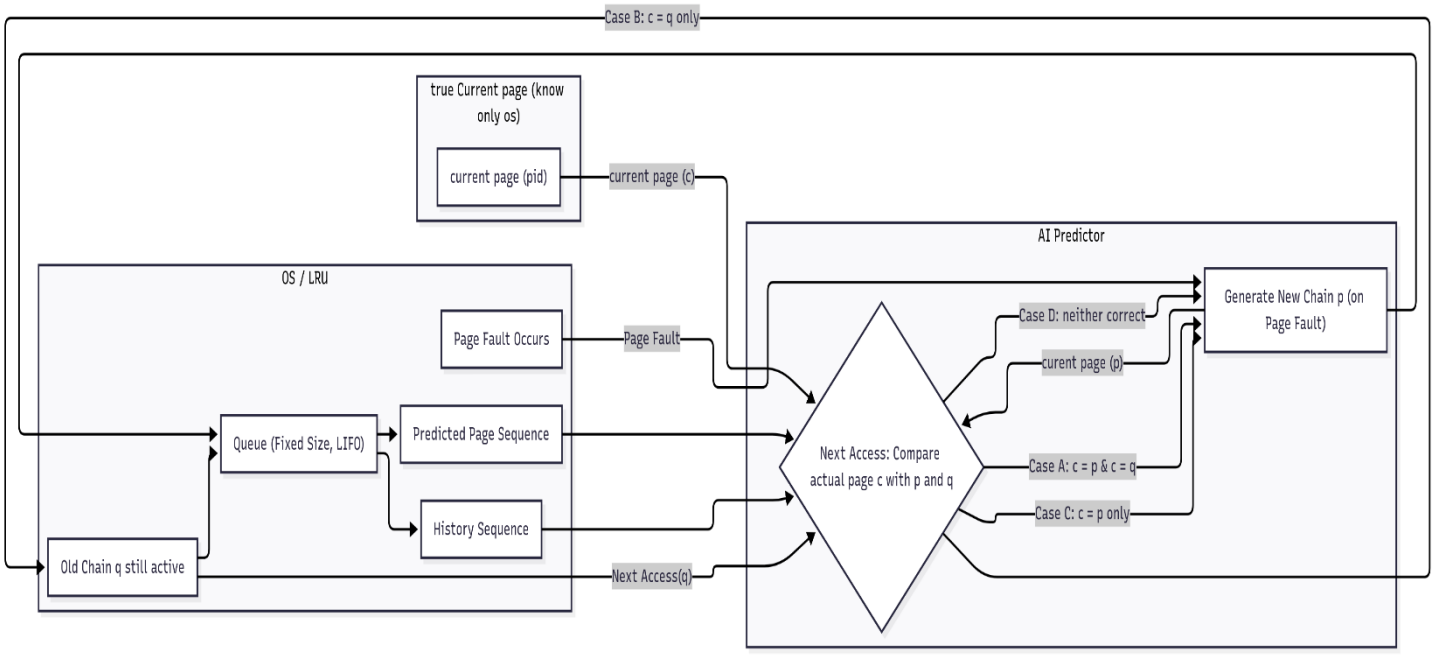
7) Page fault handling & one-shot chain generation

When OS hits a page fault:

1. OS triggers disk read; **I/O wait begins**.
2. Model is activated and generates p (one-shot chain) plus confidences.
3. Both q (old) and p (new) coexist until next accesses validate them.
4. Optionally set a small TTL for q before it’s considered stale.

---

8) Old vs New chain validation — Cases A–D (the key decision)



On the **next actual page access** c, compare c with the heads of both chains:

- Let c = actual next page. Let q_head = q[0] (if exists) and p_head = p[0].

Case A: c == q_head AND c == p_head

- Both predicted the same actual page. → **Reward both**; advance both chains. Possibly increase W slightly.

Case B: c == q_head AND c != p_head

- Old chain is correct, new chain is wrong. → **Keep old q**, advance it; mild penalty to new chain's head p_head and reduce W or S slightly.

Case C: c == p_head AND c != q_head

- New chain is correct while old chain missed. → **Accept new p**: replace q with remainder of p (or merge); reward p entries and update mu / S.

Case D: c != p_head AND c != q_head

- Both wrong → heavy penalty. OS falls back to LRU eviction and actual fault handling; **but accept the new p as the most recent chain** (with penalized W), because it's the most recent model output.

(These are exactly the four cases you outlined in DeepSeek.)

---

9) Queue updates (how q, p, and OS memory queue change)

Below is the canonical behavior (simplified):

- **On Case A**: q.pop(0), p.pop(0) — both advance. OS memory queue unchanged. Reward increase for mu[c] and S.
- **On Case B**: q.pop(0) (advance old). p remains but mu[p_head] penalized. W slightly decreased. OS memory queue unchanged.
- **On Case C**: Accept p → set q = p[1:] (advance p) and discard old q. Reward mu[p_head]. Possibly update OS hold queue if conf high.
- **On Case D**: Both wrong → OS will perform LRU eviction (a real page fault). After eviction, accept p as q (but with reduced trust). W gets significant penalty.

LIFO swap / Input queue behavior (from your notes): when a predicted head gets matched, the input queue can be updated in LIFO style — drop its last index and push the current page as the newest element. This keeps the history window relevant for next predictions.

---

10) Formal update rules (equations) — reward & penalty (use in slides)

Use simple, stable update formulas — easy to explain and implement.

Per-page fuzzy μ update (for page x):

- If prediction for x was **correct**:
- mu[x]_new = mu[x]_old + mu_reward_r * (1 - mu[x]_old)

(Example: mu_old = 0.8, mu_reward_r = 0.1 → mu_new = 0.8 + 0.1*(0.2) = 0.82)

- If prediction for x was **incorrect**:
- mu[x]_new = mu[x]_old - mu_penalty_p * mu[x]_old

(Example: mu_old = 0.7, mu_penalty_p = 0.3 → mu_new = 0.7 - 0.3*0.7 = 0.49)

Short-term score S update (fast, immediate):

- If correct:
- S_new = S_old + S_alpha * (1 - S_old)
- If incorrect:
- S_new = S_old - S_alpha * S_old

Reality weight W (daemon / slow-EMA):

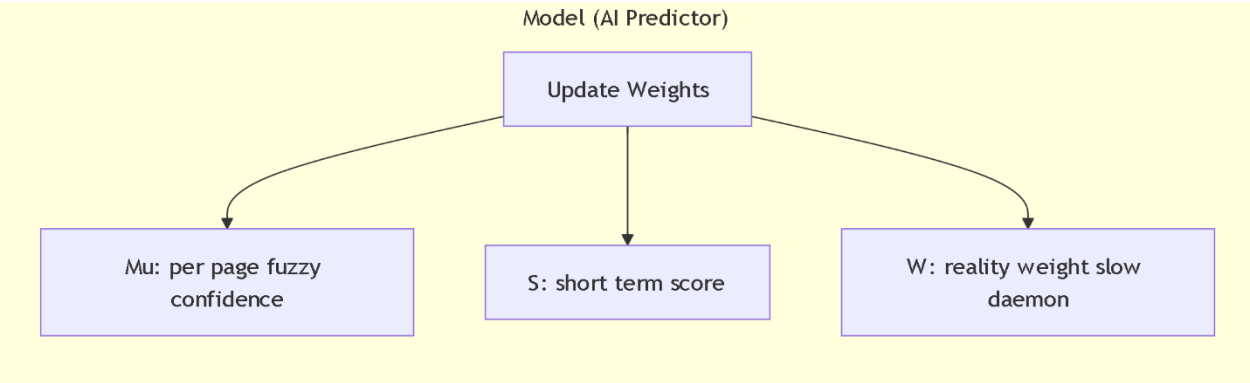
- Background daemon computes success ratio succ_ratio = successful_hold_suggestions / total_hold_suggestions over a window and updates:
- W_new = (1 - W_alpha) * W_old + W_alpha * succ_ratio

(W_alpha small like 0.05–0.10)

Alternatively the simplified on-event update:

- on success: W = W + W_alpha * (1 - W)
- on failure: W = W - W_alpha * W

Final trust function (Trust used by OS when deciding to hold):
One simple form:
Trust = w1 * S + w2 * avg(mu of predicted pages) + w3 * W
with weights w1+w2+w3 = 1 (e.g., w1=0.3, w2=0.4, w3=0.3). OS holds if Trust > trust_threshold.



---

11) Pseudocode — compact (core functions)

pseudocode (high level)
def on_page_access(c):
if q: # active predicted chain
if c == q.head():
reward_model_for(c)
q.pop_head()
update_history_push(c)
return # fast path: no LRU
else:
mismatch without fault
penalize_trust_small()
update_history_push(c)
keep q running
return
else no active chain -> normal behavior
if page_in_memory(c):
update_history_push(c)
return
else:
true page fault
on_page_fault(c)

```
def on_page_fault(missed_page):
    # OS begins IO -> use wait time to generate new chain
    p, confidences = model.generate_chain(history, mu_subset, S, W, chain_len)
    # keep old q and new p until next access validates them
    schedule_compare_next_access(p, q)

def compare_pq_on_next_access(c, p, q):
    p_head = p.head if p else None
    q_head = q.head if q else None

    if c == p_head and c == q_head:
        # Case A
        reward_page(c)
        p.pop_head(); q.pop_head()
    elif c == q_head:
        # Case B
        reward_page(c); q.pop_head()
        penalize(mu[p_head])
    elif c == p_head:
        # Case C
        reward_page(c)
        q = p[1:]  # accept new chain
    else:
        # Case D
        heavy_penalty()
        # fallback to LRU; but accept new chain p as q (with penalized W)
        q = p[1:]
        perform_lru_eviction_for(missed_page)
```

---

12) Worked numeric example (walkthrough)

Start values:

- history = [5,2,8,3,1]
- mu[3] = 0.80, mu[4] = 0.70 (just examples)
- S = 0.80, W = 0.70
- mu_reward_r = 0.10, mu_penalty_p = 0.30, S_alpha=0.15, W_alpha=0.10

Event: Page fault happened; Model generates p = [4,9,7]. Old q = [3,1,4] was active.

Next actual access c = 3 arrives.

Compare:

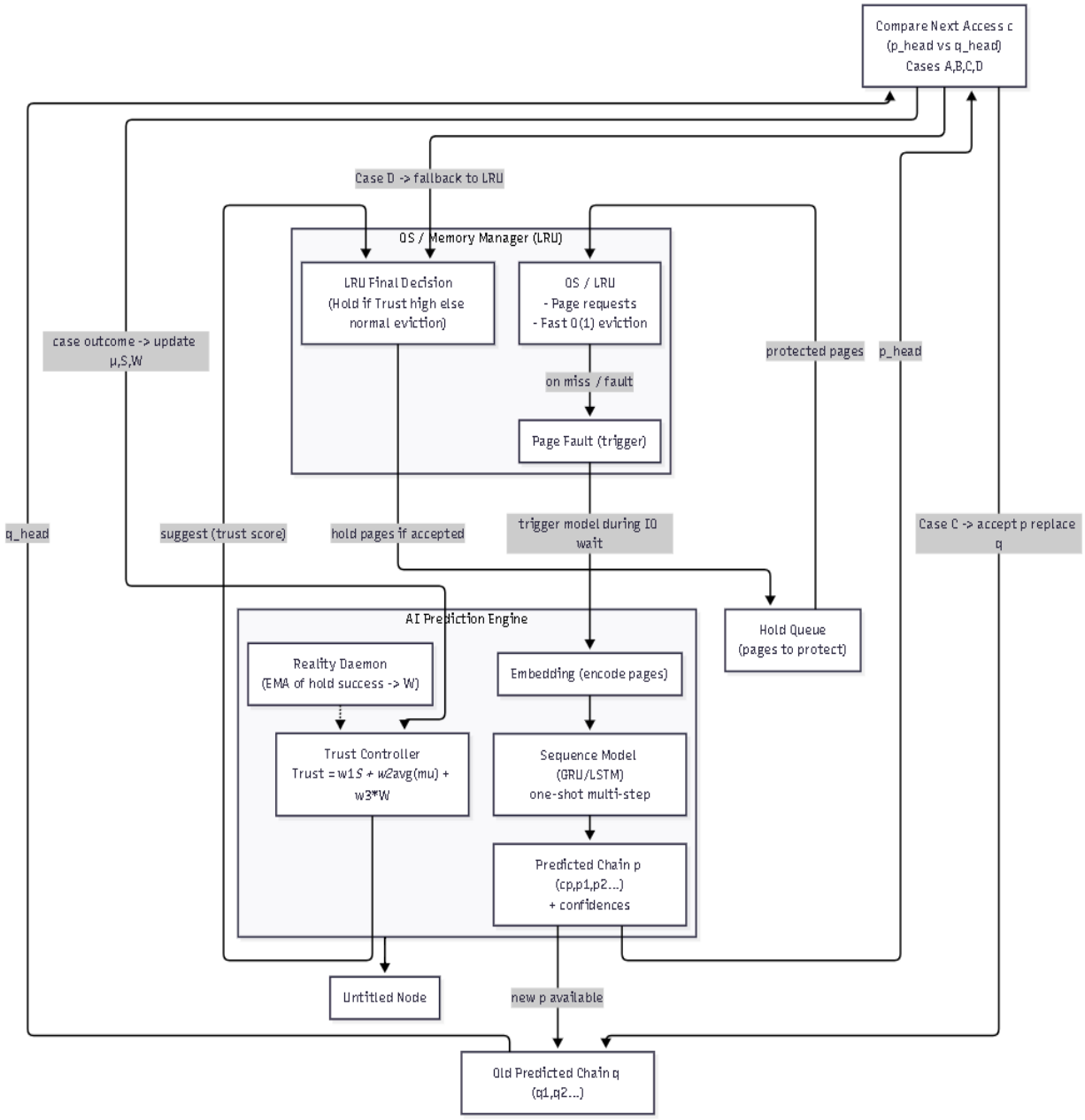
- q_head = 3 → matches q.
- p_head = 4 → does not match p.

So **Case B** (only q correct).

Updates:

1. Reward page 3:
    - mu[3]_new = 0.80 + 0.10*(1 - 0.80) = 0.80 + 0.02 = 0.82
2. Penalize p_head (page 4):
    - mu[4]_new = 0.70 - 0.30*0.70 = 0.70 - 0.21 = 0.49
3. Update S (short term):
    - Since q matched, S increases a bit:
      S_new = 0.80 + 0.15*(1 - 0.80) = 0.80 + 0.15*0.20 = 0.80 + 0.03 = 0.83
4. Update W (soft decrease for new chain wrong):
    - We do a small reduction: W_new = W_old - 0.05 * W_old = 0.70 - 0.035 = 0.665 (or via more formal rule)
5. Queue updates:
    - q advances: new q = [1,4]
    - p stays [4,9,7] but its head 4 now has lower mu.

Interpretation to audience: model correctly predicted old chain element; new chain was off. System penalizes the new chain's earliest predictions and reduces global trust slightly — but chain continues to exist for future validation. This illustrates safe, incremental integration (model not trusted blindly).


Compare Next Access c
(p_head vs q_head)
Cases A,B,C,D
Case D -> fallback to LRU
OS / Memory Manager (LRU)
LRU Final Decision
(Hold if Trust high else
normal eviction)
OS / LRU
- Page requests
- Fast O(1) eviction
on miss / fault
Page Fault (trigger)
case outcome -> update
μ,S,W
protected pages
p_head
q_head
suggest (trust score)
hold pages if accepted
trigger model during IO
wait
Case C -> accept p replace
q
AI Prediction Engine
Reality Daemon
(EMA of hold success -> W)
Embedding (encode pages)
Hold Queue
(pages to protect)
Trust Controller
Trust = w1*S + w2*avg(mu) +
w3*W
Sequence Model
(GRU/LSTM)
one-shot multi-step
Predicted Chain p
(cp,p1,p2...)
+ confidences
Untitled Node
new p available
Old Predicted Chain q
(q1,q2...)

Final Model Architectures:

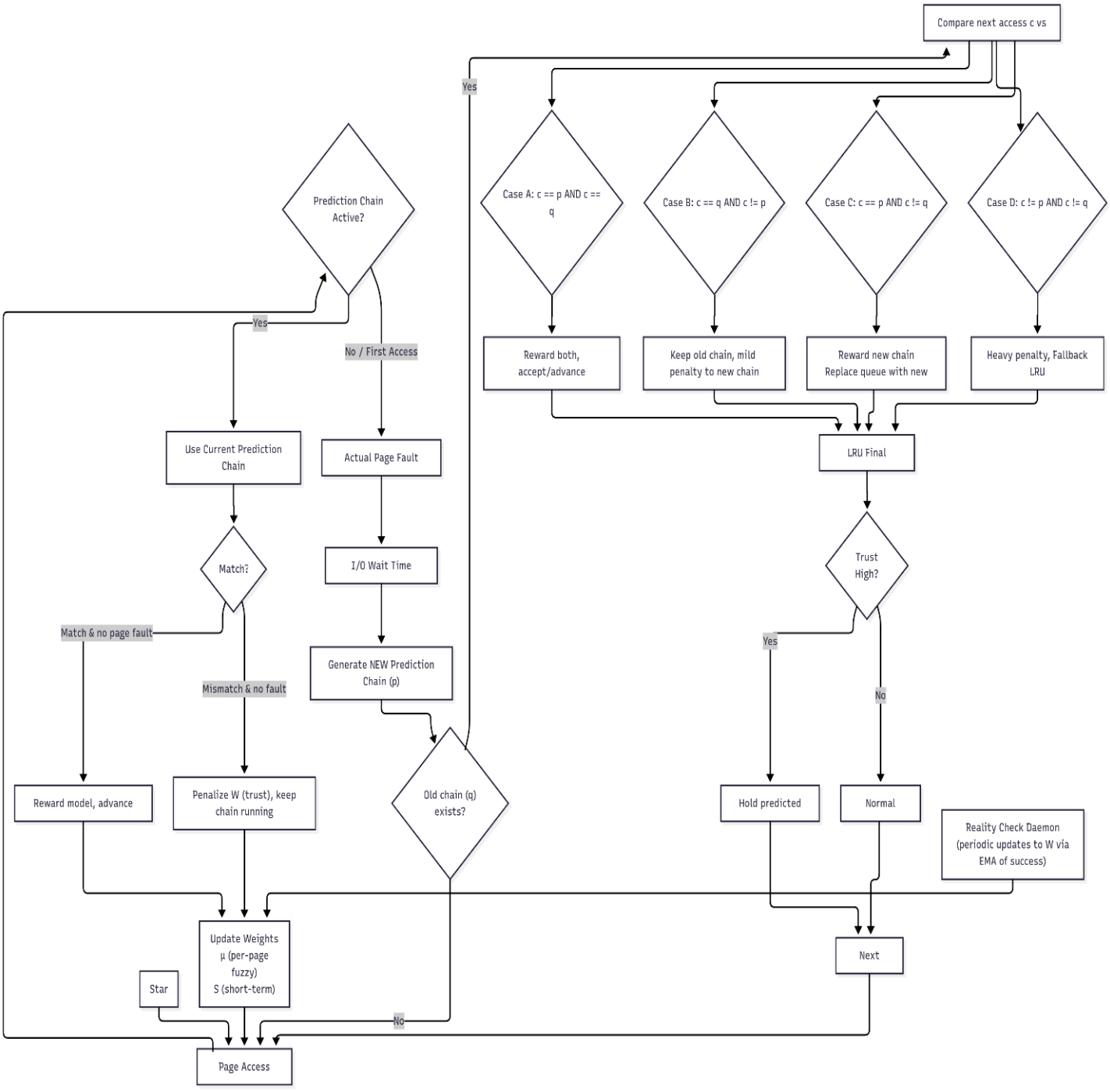


Fig. Hybrid AI-Assisted Page Replacement Architecture